CE 502 · ADVANCED TOPICS IN DATABASE SYSTEMS

Week 04

Distributed Database Systems

What changes when one database lives at many sites, what the system must hide from its users, and the algebra we use to split and rebuild tables.

THIS WEEK

- 4-1 **What a Distributed Database Is** — 45 minutes
- 4-2 **Transparency and Architecture** — 45 minutes
- 4-3 **Relational Algebra for Distribution** — 45 minutes
- P4 **Practice: a two-site database** — 90 minutes · PostgreSQL lab

Dr. Afan HASAN · [Semester]

Recap: Transactions and Concurrency

- A **transaction** is a unit of work that must be atomic, consistent, isolated and durable (ACID).
- **Locking** and isolation levels stop concurrent transactions from producing lost updates, dirty reads and phantoms.
- A **deadlock** is a cycle of transactions waiting for each other; the DBMS detects it and aborts a victim.

THIS WEEK: THE SAME IDEAS, MANY SITES

1. **Atomicity** across sites needs a commit protocol.
2. **Deadlocks** can span sites, so no single site sees the whole cycle.

Key point: everything in Weeks 1–3 assumed one server. This week we ask what changes when the data lives at several sites.

CE 502 · WEEK 4 · LESSON

4-1

What a Distributed Database Is

Lesson 1 of 3

In this lesson

01 What a distributed database is, and is not

02 Shared-memory, shared-disk and shared-nothing systems

03 Why organisations distribute their data

04 The problems every distributed DBMS must solve

45 minutes

Builds on Weeks 1–3: one database, one server

Objectives

By the end of this lesson, you will be able to:

01 **Define** a distributed database and a distributed DBMS

Checked in: Quiz 4 · question 1

02 **Distinguish** a distributed database from files, multiprocessors and centralised systems

Checked in: Quiz 4 · questions 2–3

03 **Explain** why organisations distribute and replicate data

Checked in: Quiz 4 · question 4 and the scenario task

04 **Identify** the main problems a distributed DBMS must solve

Checked in: Quiz 4 · question 5

The chip in the top-right corner of each slide shows which objective that slide serves.

Objective 03

Why Distribute a Database?

- Many organisations are **already distributed**: branches, factories and offices in different cities create and use their own data.
- Keeping each piece of data **close to the people who use it** makes most queries local, fast and cheap.
- A failure at one site should **not stop** the others, and capacity should grow by **adding machines**.

Four reasons to distribute

- Locality of reference
- Availability and reliability
- Scalability: scale out
- Local autonomy and control

Key point: distribution adds cost and complexity. It is worth it only when these benefits matter for the organisation.

Objective 02

What a Distributed Database Is Not

A collection of files

Files on a network drive are **not managed** as one database.

No common schema, no queries across them, no transactions.

A multiprocessor computer

Many processors in **one box** share memory or disk.

The data is not at geographically separate sites.

A central site on a network

All data sits on **one machine**; the others only connect to it.

That is remote access to a centralised database.

⚠ **Common mistake:** calling any system with several computers “distributed”. In this course, the **data itself** must be stored at separate sites and managed as one database.

Objective 02

Shared Memory, Shared Disk and Shared Nothing

Shared memory

Processors share **memory and disks**.

Simple to program; the memory bus becomes the bottleneck as processors are added.

Shared disk

Each processor has its own memory; all share the **disks**.

Needs coordination of caches; the disk subsystem limits growth.

Shared nothing

Each node has its **own memory and disks**.

Nodes talk only by messages; scales to hundreds of nodes.

In practice

Most large analytical systems today are shared nothing: for example Greenplum, Amazon Redshift, Citus and Apache Spark clusters. Oracle RAC is a well-known shared-disk design.

Objective 01

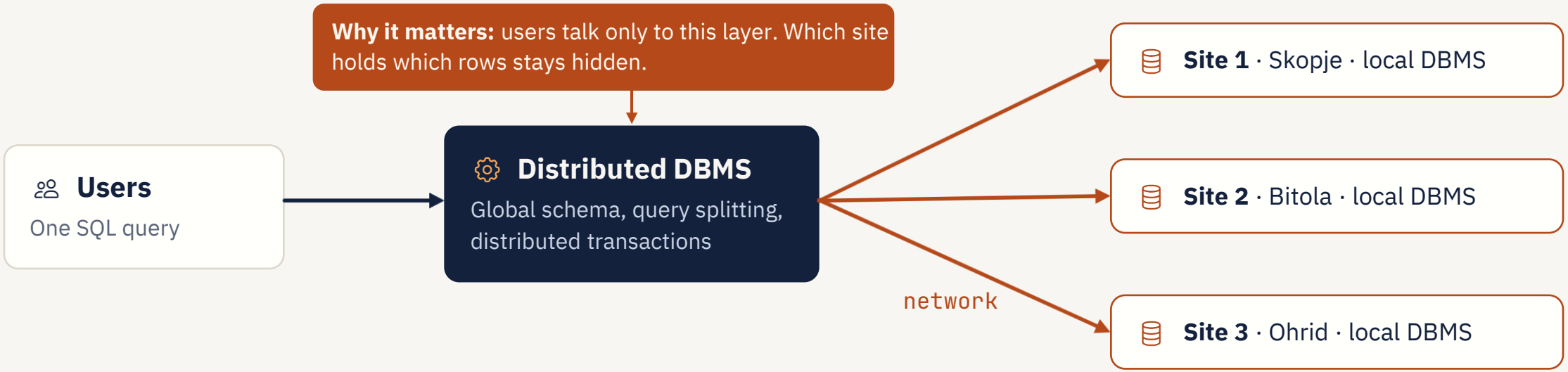
A distributed database is related data at separate sites, managed as one database.

Formally: a collection of multiple, logically interrelated databases distributed over a computer network. A distributed DBMS is the software that manages it and makes the distribution transparent to users.

- Test any system with three questions: related data? separate sites? one managed database?

Objective 01

The Distributed DBMS Hides the Sites



- Each site runs its **own local DBMS** with its own log, locks and storage.
- The **distributed DBMS** layer turns one user request into work at each site and combines the results.

Objective 03

Running Example: One Company, Four Sites

Site	Stores	Why here?
Baltimore	Maryland customers · Maryland manufacturers	Most queries about MD come from Baltimore
New York	New York customers · NY manufacturers · copy of MD customers	NY sales staff often serve MD customers
Chicago	Illinois customers · local tables	Local office
Los Angeles	California customers · local tables	Local office

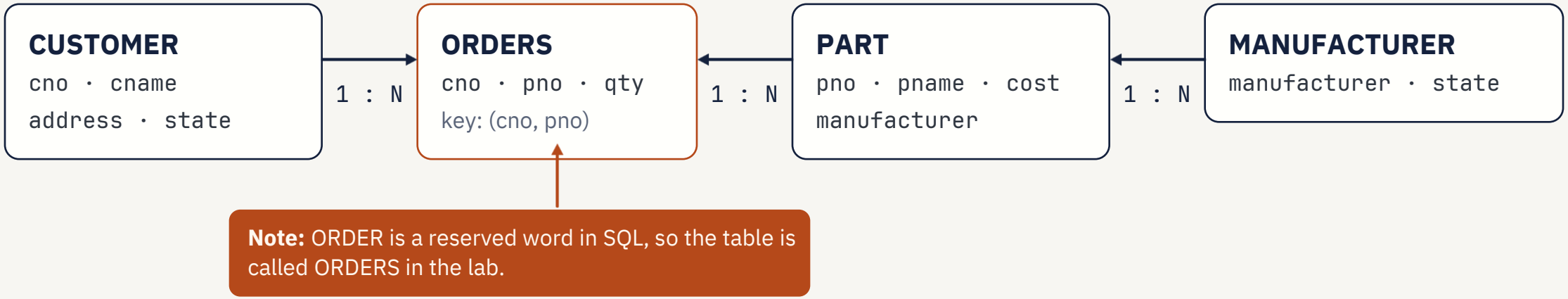
← **Read this row first.** The copy in New York exists for two reasons: availability and locality of reference.

If Baltimore is down, New York can still answer questions about Maryland customers.

⚠ **Common mistake:** assuming every table should be copied everywhere. Each copy must be updated on every change, so copies pay off only for data that is read far more often than it is written.

Objective 01

The Schema Used All Week



- One customer places many orders; one part appears in many orders.
- One manufacturer makes many parts; each manufacturer has a home **state**.

Objective 03

The Price of a Copy

Operation	One copy	Copy at every site
Read at a remote site	Crosses the network	Local and fast
Read when one site is down	May fail	Still works
Update	One site	Every site, kept consistent
Storage	Once	Once per site

Why? Every copy must apply every update, inside one distributed transaction or soon after it.

In practice

Small, read-mostly tables such as PART and MANUFACTURER are usually **fully replicated**. Large, often-updated tables are not.

Key point: replicate what is read often and changed rarely; keep frequently updated data at one site.

Objective 04

Problems Every Distributed DBMS Must Solve

Problem	The question	In CE 502
Database design	How to split tables into fragments and where to place them	Weeks 5–6
Query processing	Which sites to use and in what order, to ship the fewest bytes	Lesson 4-3 (overview)
Directory management	Where to keep the global schema and the location of every fragment	Lesson 4-2
Concurrency and deadlocks	How to lock across sites; how to find a cycle no single site can see	Week 3, Week 9
Reliability	How to commit at all sites or none, despite failures	Weeks 1–2, next slide
Heterogeneity	How to integrate databases built on different products	Lesson 4-2

Key point: finding the **optimal** design and the optimal query plan are both NP-hard, so real systems use heuristics.

Objective 04

Two-Phase Commit: All Sites or None

01

Prepare?

The coordinator asks every participant site to **prepare**: write its changes and a PREPARE record to its log.

02

Vote

Each site answers **YES** (ready to commit) or **NO** (must abort).

03

Decide

All YES: the coordinator logs **COMMIT**. Any NO or no answer: it logs **ABORT**.

04

Finish

The decision is sent to every site, which commits or aborts and acknowledges.

Key point: a site that has voted YES must wait for the decision; it can no longer abort on its own.

Objective 03

Scenario: Where Do the Texas Customers Go?

THE SITUATION

The company opens a new office in Dallas but will not add a database site there yet. A new table, **Texas customers**, must be placed at one of the existing sites.

Measured over a month:

> “70% of the queries on Texas customers come from **Los Angeles** and 30% from **Chicago**. About 5% of the requests are updates.”

YOUR TASK · PAIRS · 6 MIN

1. At which site would you store the table? Why?
2. Would you also keep a copy at the other site?
3. What extra cost does that copy bring?
4. What would change if half of the requests were updates?

Objective 02

Check Your Understanding

Which of these is a **distributed database** in the sense of this course?

A A server with 64 processors sharing memory and disks

B Related databases in Skopje and Bitola, queried as one database

C A shared network drive full of CSV files

D Ten offices whose programs all connect to one central server

Think for 30 seconds, then vote with your fingers: 1 = A, 2 = B, 3 = C, 4 = D.

Key Terms

Distributed database

A collection of logically related databases distributed over a computer network.

Site

A computer with its own local DBMS and storage, connected to the others by a network.

Locality of reference

Storing data at the site where it is used most often.

Two-phase commit (2PC)

A protocol in which all sites vote, then all commit or all abort.

Distributed DBMS (DDBMS)

Software that manages a distributed database and hides the distribution from users.

Shared nothing

Each node has its own processor, memory and disks; nodes cooperate only by messages.

Replication

Storing copies of the same data at more than one site.

Scale out

Adding machines to grow capacity, instead of buying a bigger machine.

Summary

In this lesson, you learned how to:

- **01** Define a distributed database and a distributed DBMS
- **02** Distinguish a distributed database from files, multiprocessors and centralised systems
- **03** Explain why organisations distribute and replicate data
- **04** Identify the main problems a distributed DBMS must solve

Key takeaway: distribute data where it is used, copy only what is read far more than it is written, and let the DDBMS hide the rest.

Next lesson

4-2 Transparency and Architecture

CE 502 · WEEK 4 · LESSON

4-2

Transparency and Architecture

Lesson 2 of 3

In this lesson

01 What transparency means and its six kinds

02 The ANSI/SPARC levels and the global schema

03 Classifying systems: autonomy, distribution, heterogeneity

04 Client–server, peer-to-peer and multi-database systems

45 minutes

Builds on Lesson 4-1: the distributed DBMS

Objectives

By the end of this lesson, you will be able to:

01 **Explain** the six kinds of transparency a distributed DBMS provides

Checked in: Quiz 4 · questions 6–7

02 **Describe** the ANSI/SPARC levels and the global and local schemas

Checked in: Quiz 4 · question 8

03 **Classify** systems by autonomy, distribution and heterogeneity

Checked in: Quiz 4 · questions 9–10 and the scenario task

04 **Compare** client–server, peer-to-peer and multi-database architectures

Checked in: Quiz 4 · question 11 and the design question

The chip in the top-right corner of each slide shows which objective that slide serves.

Objective 01

What Is Transparency?

- **Transparency** means that something exists but the user does not see it and does not need to manage it.
- A distributed DBMS aims to give users the view of **one ordinary database**, whatever happens below.
- The query on the right touches four tables at several sites, yet it names **no site** at all.

```
-- one global query
SELECT c.cname, p.pname, o.qty
FROM customer c
JOIN orders o ON o.cno = c.cno
JOIN part p ON p.pno = o.pno
JOIN manufacturer m ON m.manufacturer = p.manufacturer
WHERE m.state = 'MD';
```

← **Note:** no site names, no copies, no fragments. The DDBMS finds them.

Objective 01

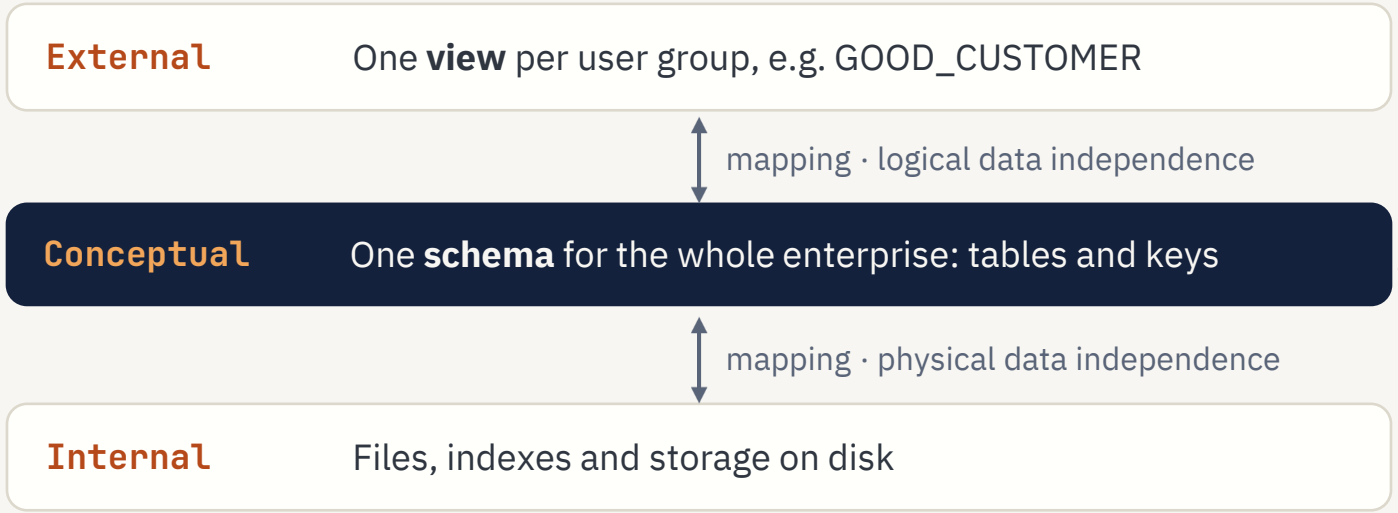
Six Kinds of Transparency

Transparency	The user does not need to know...	Example
Logical data independence	that the schema changed, e.g. a column was added	Old queries still run
Physical data independence	how data is stored: files, indexes, media	A new index changes no code
Location (distribution)	at which site a table or fragment is stored	No site names in SQL
Network	the network topology, protocols or site-specific names	A link is rerouted silently
Replication	how many copies exist and how they are kept consistent	Update once, all copies change
Fragmentation	that a table is split into horizontal or vertical fragments	Query CUSTOMER, not CUSTOMER_MD

⚠ **Common mistake:** thinking replication transparency is only a convenience. If users had to update every copy themselves, keeping copies consistent would be practically impossible.

Objective 02

ANSI/SPARC: Three Levels, Two Mappings



Critical point: in a distributed database there is still **one** conceptual schema, but **many** internal ones, one per site.

- Each level is mapped to the next. A request travels **down** through the mappings; the answer travels back **up**.

Objective 02

Example: An External View on the Conceptual Schema

```
-- external level, defined on the conceptual level
CREATE VIEW good_customer AS
SELECT c.cno, c.cname, count(*) AS n_orders
FROM customer c
JOIN orders o ON o.cno = c.cno
GROUP BY c.cno, c.cname
HAVING count(*) > 10;
```

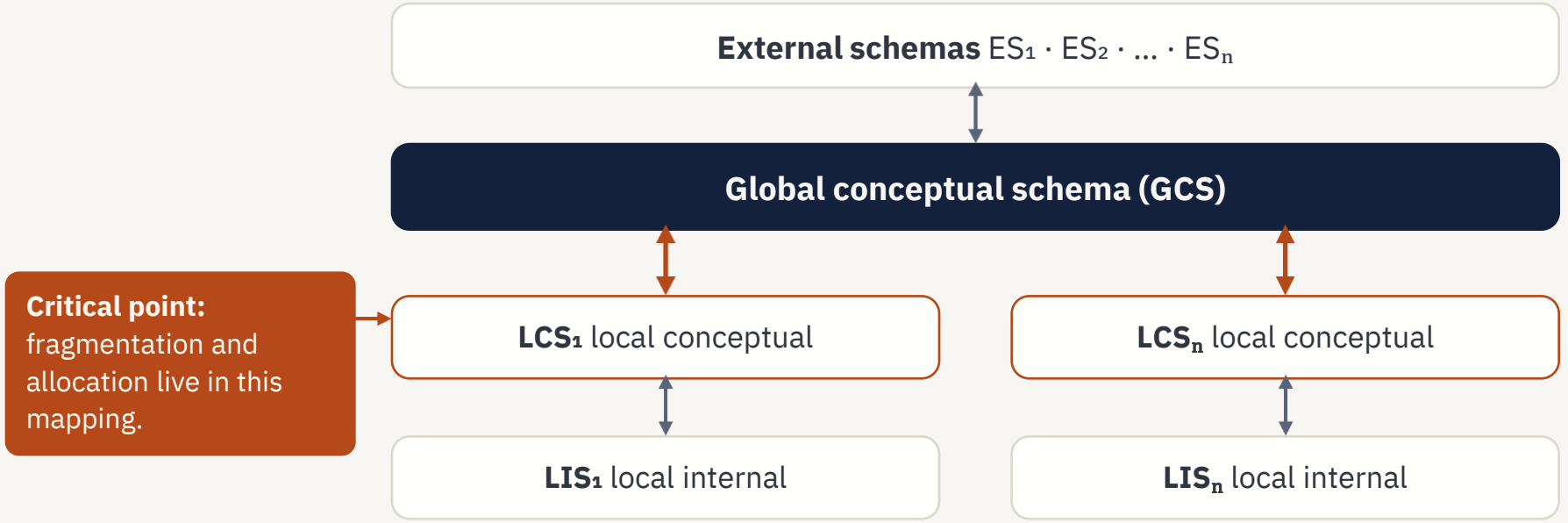
1. The view is the **external** schema of the sales team.
2. CUSTOMER and ORDERS belong to the **conceptual** schema.

← **Why?** If ORDERS gets a new column, this view and its users are unaffected: logical data independence.

3. Indexes on orders.cno are part of the **internal** schema.

Objective 02

Reference Architecture: Global and Local Schemas



Objective 03

Three Independent Dimensions

Autonomy

A0 tight integration

A1 semi-autonomous

A2 total isolation

How independently does each site decide, operate and join or leave?

Distribution

D0 not distributed

D1 client–server

D2 peer-to-peer

Where is the data, and does every site have full DBMS functionality?

Heterogeneity

H0 homogeneous

H1 heterogeneous

Do the sites differ in hardware, DBMS product, data model or query language?

Key point: choose one value on each axis independently. Every combination describes a possible architecture.

Objective 03

Combinations and Today's Systems

Combination	What it is	A system today
A0 · D2 · H0	Classic distributed DBMS	Citus cluster, CockroachDB
A1 · D0 · H1	Federated system on one machine	One server combining text and image stores
A2 · D2 · H1	Peer-to-peer multi-database system	Trino over MySQL, PostgreSQL and S3
A0 · D1 · H0	Client–server DBMS	One PostgreSQL server, many clients

← **Read this row first.** CE 502 designs systems like this one: top-down, with full control.

The right-hand column is our own mapping. Borderline cases are normal; the justification counts.

Key point: the axes classify **architectures**, not brands: one product can be deployed in different ways and land in different cells.

Objective 03

The more autonomous the sites, the less transparency the system can give.

A tightly integrated system controls every site and can hide everything. A federation of independent databases can only hide what their owners agree to share.

Rule of thumb: full transparency needs top-down design; bottom-up integration gives partial transparency.

Client–Server: Who Combines the Data?

Variant 1 · one server

Many clients, **one server** with all database functions.

Simplest; the server can become a bottleneck.

Variant 2 · thick clients

Each client connects to **many servers** and integrates the data itself.

Every application repeats complex work.

Variant 3 · middle tier

Light clients call a **server that routes** requests to other servers.

Integration is done once, in the middle.

In practice

Variant 3 is the usual pattern today: a coordinator or router in front of many data nodes, such as the Citus coordinator, CockroachDB gateways or the mongos router in MongoDB.

Objective 04

Distributed DBMS or Multi-Database System?

	Distributed DBMS	Multi-database (MDBS)
Design	Top-down, full control	Bottom-up from existing databases
Autonomy	Low (A0)	High (A1–A2)
Global schema	Describes all data	Only shared data, or none
Existing users	None yet	Must keep working unchanged
Transparency	Full	Partial

Note: without a global schema, users must know every source and write the mappings themselves.

In practice

Trino and PostgreSQL's **postgres_fdw** are modern multi-database tools.

⚠ **Common mistake:** calling every federation tool a distributed DBMS. If the sources stay autonomous and were built separately, it is a multi-database system.

Objective 03

Scenario: Linking Two University Systems

THE SITUATION

A university has a library system on **Oracle Database** and a student system on **Microsoft SQL Server**. Both are owned by different departments.

The rector’s office wants one report of borrowed books per student, every week.

“Neither system may be changed, and both must keep running exactly as they do now.”

YOUR TASK · PAIRS · 7 MIN

1. Classify the integrated system: A? D? H?
2. Distributed DBMS or multi-database system? Why?
3. Which transparency can you offer? Which not?
4. Which client–server variant fits best?

Objective 01

Check Your Understanding

A DBA creates an index on **orders.cno**. Queries become faster, and no application has to change. Which kind of transparency does this show?

A Location transparency

B Physical data independence

C Logical data independence

D Fragmentation transparency

Think for 30 seconds, then vote with your fingers: 1 = A, 2 = B, 3 = C, 4 = D.

Key Terms

Transparency

Hiding a property of the system so that users need not know or manage it.

Location transparency

Users need not know at which site data is stored.

Global conceptual schema

One schema describing the whole distributed database, as if stored at one site.

Heterogeneity

Differences between sites in hardware, DBMS, data model or language (H0–H1).

Data independence

Applications are unaffected by changes to the logical (schema) or physical (storage) level.

Replication transparency

Users need not know how many copies exist or keep them consistent.

Autonomy

The degree to which each site operates and decides independently (A0–A2).

Multi-database system

An integration of existing, autonomous databases, built bottom-up.

Summary

In this lesson, you learned how to:

- **01** Explain the six kinds of transparency a distributed DBMS provides
- **02** Describe the ANSI/SPARC levels and the global and local schemas
- **03** Classify systems by autonomy, distribution and heterogeneity
- **04** Compare client–server, peer-to-peer and multi-database architectures

Key takeaway: a distributed DBMS hides location, fragmentation and copies behind one global schema; the more autonomous the sites, the less it can hide.

Next lesson

4-3 Relational Algebra for Distribution

CE 502 · WEEK 4 · LESSON

4-3

Relational Algebra for Distribution

Lesson 3 of 3

In this lesson

01 Why distribution needs relational algebra

02 Normalised tables: the unit we fragment

03 Selection, projection, union, difference, product

04 Joins and semi-joins, and translating SQL

45 minutes

Builds on Lesson 4-2: the global schema

Objectives

By the end of this lesson, you will be able to:

01 **Explain** why distribution designs start from normalised tables

Checked in: Quiz 4 · question 12

02 **Apply** selection, projection, union, difference and product

Checked in: Quiz 4 · questions 13–14

03 **Apply** joins and the semi-join, and say which operator rebuilds which fragments

Checked in: Quiz 4 · question 15

04 **Translate** multi-table queries between SQL and relational algebra

Checked in: Quiz 4 · question 16 and Practice 4 · Part A

The chip in the top-right corner of each slide shows which objective that slide serves.

Objective 02

Why Distribution Needs Relational Algebra

- **Fragments are defined** with algebra: a horizontal fragment is a selection σ, a vertical fragment is a projection π.
- **Tables are rebuilt** with algebra: union ∪ for horizontal fragments, join ⋈ for vertical ones.
- **Distributed queries are planned** as algebra trees, with selections pushed down to the sites.

PREVIEW OF WEEKS 5-6

```
CUSTOMER_MD = σ state='MD' (CUSTOMER)

CUSTOMER_NY = σ state='NY' (CUSTOMER)

CUSTOMER = CUSTOMER_MD U CUSTOMER_NY
```

Key point: σ and π cut a table into fragments; ∪ and ⋈ put it back together.

Objective 01

Why We Fragment Normalised Tables

pno	pname	cost	manufacturer	state
1	widget	3	Acme Inc.	MD
2	thingamabob	5	Acme Inc.	MD
3	doodad	15	XYZ Enterprises	NY

Four anomalies

- **Redundancy:** Acme's state repeats on every Acme part
- **Update:** Acme moves; change every Acme row
- **Insertion:** no new maker without a part
- **Deletion:** delete the doodad, lose XYZ

← **Why it matters here:** every row we fragment carries this redundancy to every site. Normalise first, then distribute.

The Fix: Decompose, Then Rebuild With a Join

PART

pno · pname · cost · manufacturer

One row per part.

MANUFACTURER

manufacturer · state

One row per manufacturer: each state stored **once**.

Rebuild

PART ⋈ MANUFACTURER

The natural join on **manufacturer** gives back the original table, no more and no fewer rows.

In practice

A decomposition is **lossless** when the join of the pieces gives back exactly the original rows. Joining on a key (here manufacturer) guarantees this.

Objective 02

Five Fundamental Operations

Operation	Algebra	SQL
Selection (rows)	σ cost > 10 (PART)	SELECT * FROM part WHERE cost > 10
Projection (columns)	π pname, cost (PART)	SELECT DISTINCT pname, cost FROM part
Union	R ∪ S	R UNION S
Difference	R − S	R EXCEPT S
Cartesian product	R × S	SELECT * FROM r CROSS JOIN s

⚠ **Common mistake:** forgetting DISTINCT. The algebra works on sets, so π removes duplicate rows; SQL's SELECT keeps them unless you write DISTINCT.

Objective 02

Worked Example: Selection, Then Projection

`π pname, cost (σ cost > 10 (PART))`

pname	cost
tool chest	15.00
sensor	22.00
controller	48.00

Result on the Practice 4 data: 3 of the 8 parts.

1. Evaluate the **inner** expression first: keep parts with cost above 10.
2. Then keep only the columns **pname** and **cost**.

← **Read this first:** the order matters for cost, not for the result. Filtering early means fewer rows to project, or to ship.

In practice

In Practice 4 you will see PostgreSQL send the condition of a query on the global view to each site inside its **Remote SQL**: the selection is pushed down to the data.

Objective 02

Union, Difference and Product

Union and difference

R = {widget, thingamabob}

S = {thingamabob, doodad}

R ∪ S = {widget, thingamabob, doodad}

R − S = {widget}

Cartesian product

R has 2 rows, S has 3 rows.

R × S has 2 × 3 = **6 rows** and all columns of both.

CUSTOMER × MANUFACTURER on the lab data: 9 × 4 = 36 rows.

Key point: union and difference need **union-compatible** tables: the same number of columns, with matching domains, in the same order.

Objective 03

Derived Operations We Will Use a Lot

Intersection

R ∩ S = R − (R − S)

Rows in both. SQL: INTERSECT.

Theta join

R ⋈θ S = σθ (R × S)

Any comparison, e.g. < or =.

Natural join

PART ⋈ MANUFACTURER

Equal values in **all** common columns. Rebuilds vertical fragments.

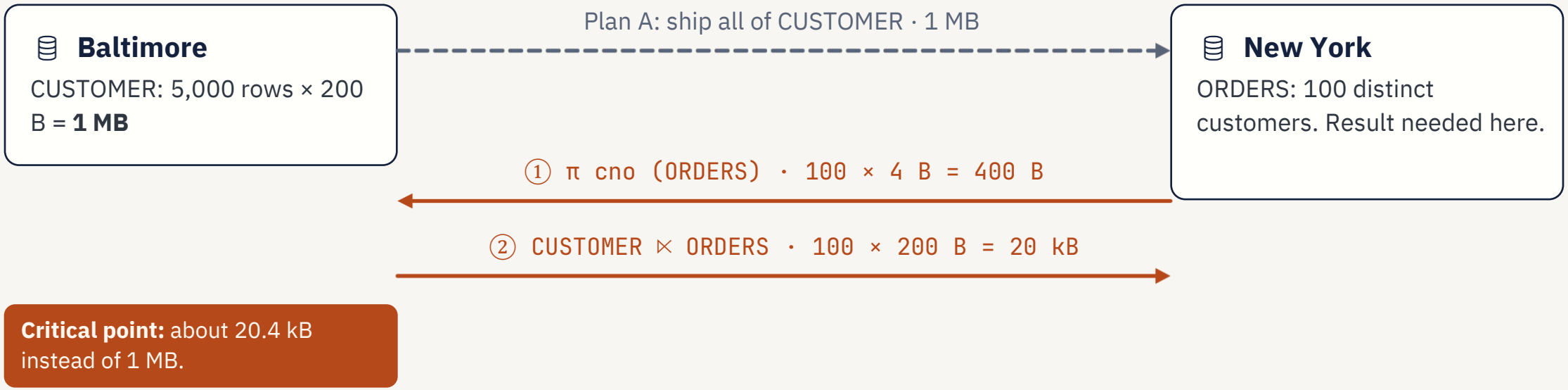
Semi-join

CUSTOMER ⋉ ORDERS

Rows of CUSTOMER that have a match. Defines derived fragments.

⚠ **Common mistake:** assuming a natural join matches only the column you had in mind. It matches **every** column with the same name in both tables.

Why the Semi-Join Matters: Ship Fewer Bytes



- Plan B sends the **join column** first, then only the rows that will actually match. It pays off when few rows match.

Horizontal fragments come back with ∪. Vertical fragments come back with ⋈.

A fragmentation is only correct if the original table can be rebuilt exactly from its fragments. Week 5 proves this for rows, Week 6 for columns.

Rule of thumb: every vertical fragment keeps the key, so the join cannot lose or invent rows.

Objective 04

Translating a Multi-Table Query

"Customer, part and quantity for parts made in Maryland"

```
-- relational algebra
π cname, pname, qty (
CUSTOMER ⋈ ORDERS ⋈ PART
⋈ π manufacturer (σ state='MD' (MANUFACTURER)) )
```

Critical: without the inner π, a natural join also matches CUSTOMER.state with MANUFACTURER.state.

On the lab data: **5 rows** with the inner π, only **3 rows** without it.

⚠ **Common mistake:** using NATURAL JOIN in SQL on real schemas. Columns that share a name by accident change the query silently; prefer JOIN … ON with explicit columns.

Objective 04

Exercise: Translate Both Ways

</> THE SITUATION

Use the schema CUSTOMER(cno, cname, address, state), ORDERS(cno, pno, qty), PART(pno, pname, manufacturer, cost), MANUFACTURER(manufacturer, state).

A · algebra → SQL

1. π cno (CUSTOMER) - π cno (ORDERS)
2. σ cost > 10 ∧ manufacturer='XYZ Enterprises' (PART)

YOUR TASK · PAIRS · 8 MIN

1–2. Write the SQL for items 1 and 2.

B · SQL → algebra

3. SELECT pname FROM part WHERE cost < 5 UNION SELECT pname FROM part WHERE manufacturer = 'Prairie Tools';

4. Customers who ordered at least 100 units of some part.

Check Your Understanding

Which operation returns the customers who have **at least one order**, with only CUSTOMER's columns and each customer **once**?

A CUSTOMER ⋈ ORDERS

B CUSTOMER ⋉ ORDERS

C CUSTOMER ∩ ORDERS

D CUSTOMER × ORDERS

Think for 30 seconds, then vote with your fingers: 1 = A, 2 = B, 3 = C, 4 = D.

Key Terms

Selection σ

Keeps the rows of a table that satisfy a condition.

Projection π

Keeps the listed columns of a table and removes duplicate rows.

Union compatibility

Same number of columns with matching domains, needed for ∪, − and ∩.

Natural join ⋈

Combines rows with equal values in all columns of the same name.

Semi-join ⋉

The rows of the left table that have a match in the right table.

Lossless decomposition

Splitting a table so that a join of the pieces gives back exactly the original.

Update anomaly

One fact stored in many rows must be changed in all of them.

Selection push-down

Applying a selection as early as possible, close to the data.

Summary

In this lesson, you learned how to:

- **01** Explain why distribution designs start from normalised tables
- **02** Apply selection, projection, union, difference and product
- **03** Apply joins and the semi-join, and say which operator rebuilds which fragments
- **04** Translate multi-table queries between SQL and relational algebra

Key takeaway: σ and π define fragments, ∪ and ⋈ rebuild tables, and ⋉ ships fewer bytes. Check every translation on real data.

Next week

Week 5 · Horizontal Fragmentation

Practice 4: Build and Query a Two-Site Database

You play the distributed DBMS: two sites, one global schema, and a look at the SQL sent to each site.

Setup	Create the database **week04** and load the course schema: 4 manufacturers, 9 customers, 8 parts, 16 orders.	–
Part A	Translate 10 items between relational algebra and SQL, and run your SQL.	Lesson 4-3
Part B	Build two sites with postgres_fdw and query them through one global schema.	Lessons 4-1, 4-2
Part C	Classify 8 real systems on autonomy, distribution and heterogeneity.	Lesson 4-2

Individual · about 90 minutes · the course PostgreSQL 16 container and the **labs/week04** folder of the course kit.

Setup + Part A

Load the Schema and Translate the Algebra

```
-- terminal (from the kit folder)
1 docker exec -it ce502-postgres psql -U postgres -d lab \
2 -c "CREATE DATABASE week04"
3 docker exec -it ce502-postgres psql -U postgres -d week04 \
4 -f /kit/labs/week04/sql/01_schema.sql
5 # write your answers in labs/week04/my_answers.sql
6 docker exec -it ce502-postgres psql -U postgres -d week04 \
7 -f /kit/labs/week04/my_answers.sql
```

1. Create the database (lines 1–2).
2. Load 4 manufacturers, 9 customers, 8 parts and 16 orders (lines 3–4).
3. Answer the 10 items of **ra-exercises.md**; save, then run (lines 5–7).

← **Check yourself:** item 4, π cno (CUSTOMER) − π cno (ORDERS), returns exactly one customer.

⚠ **Common mistake:** editing files inside the container. The kit is mounted **read-only** at /kit; write my_answers.sql on your laptop, in the kit folder.

Part B

Query Two Sites Through One Global Schema

```
-- terminal, then psql in ddb_global
1 docker exec -it ce502-postgres psql -U postgres -d lab \
2 -f /kit/labs/week04/sql/03_distributed_sim.sql
3 docker exec -it ce502-postgres psql -U postgres -d ddb_global
4 EXPLAIN (VERBOSE, COSTS OFF) SELECT * FROM customer WHERE state = 'MD';
5 INSERT INTO bal.customer VALUES (10,'Peoria Pumps','Peoria','IL');
6 INSERT INTO nyc.customer VALUES (11,'Austin Air','Austin','TX');
```

1. Create both sites and the global layer (lines 1–3).
2. Run **\d+ customer**: how is the table rebuilt?
3. Line 4: find the **Remote SQL** sent to each site.

← **Critical:** line 6 fails with a CHECK violation. The CHECK constraint **is** the fragment predicate: Texas belongs to no site.

What to Hand In

One PDF, at most four pages, submitted before the Week 5 class.

Report item	Points
Part A: my_answers.sql with output; algebra for the SQL items, with one sentence each	3
Part B: view definition, plan with Remote SQL, both INSERTs, each explained in 1–2 sentences	3
Part C: the completed A/D/H worksheet with a justification per system	2
Reflection (half a page): which transparencies ddb_global provides, and which it does not	2
Total	10

Key point: most points are for explanations: link every output to a concept from Lessons 4-1 to 4-3.

Papers to Read

Three required papers on shared-nothing systems, parallel databases and federated databases, and three optional ones.

Before the Week 5 class

Half-page note per required paper

Required

Required Reading for Week 5

The Case for Shared Nothing
M. Stonebraker · IEEE Database Engineering Bulletin 9(1), 1986, pp. 4–9

Why nodes that share nothing scale best. Read the whole paper (6 pages).

Parallel Database Systems: The Future of High Performance Database Systems
D. DeWitt, J. Gray · Communications of the ACM 35(6), 1992, pp. 85–98

Speedup, scaleup and data partitioning. Read the first half, up to data partitioning.

Federated Database Systems for Managing Distributed, Heterogeneous, and Autonomous Databases
A. Sheth, J. Larson · ACM Computing Surveys 22(3), 1990, pp. 183–236

The classic taxonomy of autonomy and federation. Read section 1.

Bring a half-page note per paper: the problem, the key idea, and its link to this week.

Optional

Going Further

Introduction to a System for Distributed Databases (SDD-1)
J. Rothnie et al. · ACM TODS 5(1), 1980, pp. 1–17

One of the first distributed DBMSs: query processing, concurrency, reliability.

Using Semi-Joins to Solve Relational Queries
P. Bernstein, D. Chiu · Journal of the ACM 28(1), 1981, pp. 25–40

The theory behind the semi-join slide of Lesson 4-3.

Principles of Distributed Database Systems, chapter 1
M. T. Özsu, P. Valduriez · Springer, 4th edition, 2020

The textbook view of this whole week: definitions, transparency, architectures.

In practice

Distributed SQL systems such as CockroachDB, YugabyteDB and Google Spanner revive the top-down distributed DBMS of the 1980s on today’s hardware.

Required

Shared Nothing Is the Most Cost-Effective Design

Stonebraker, M. (1986). The case for shared nothing. IEEE Database Engineering Bulletin, 9(1), 4–9.

THE PROBLEM

In 1986, high transaction rate systems were built as shared-memory, shared-disk or shared-nothing multiprocessors. Stonebraker asks which design is best, and whether the weak points of shared nothing really matter in practice.

KEY IDEAS

01 Rates the three designs on 12 points; shared nothing is best for scaling, bandwidth and distance

02 Shared memory cannot scale to many processors, and shared disk is best at nothing

03 Shared nothing comes almost free with a distributed DBMS, which every vendor must build

Key term: Shared nothing — Processors share neither memory nor disks; each has its own, and they cooperate only by messages

Required

Its Weak Points Matter Little in the Common Case

WHAT TO TAKE AWAY

1. Database design and load balancing are harder, but automatic design tools can partition data
2. If most transactions stay at one site (nearly delightful data), messages are not the main cost
3. Hot spots hurt every design; remove them or split each into N subrecords

THIS WEEK

Lesson 4-1's slide 'Shared Memory, Shared Disk and Shared Nothing' builds on this paper. In Practice 4, each site owns its own data, and the global layer only sends it queries.

LIMITATION

A short position paper. Table 1's ratings are the author's judgment, with no measurements, and they concern transaction processing in 1986.

For your half-page note: Stonebraker rates shared nothing worst on database design and load balancing. How does he argue that these problems are small?

Required

Parallel Databases Won With Shared Nothing

DeWitt, D., & Gray, J. (1992). Parallel database systems: The future of high performance database systems. Communications of the ACM, 35(6), 85–98. https://doi.org/10.1145/129888.129894

THE PROBLEM

In the 1980s, database machines built from special hardware failed, and critics expected I/O to limit multiprocessors. DeWitt and Gray explain why parallel database systems on ordinary hardware succeeded instead, and how they work.

KEY IDEAS

01 Linear speedup: twice the hardware does the same job in half the time

02 Linear scaleup: twice the hardware does twice as large a job in the same time

03 Relational queries form dataflow graphs, which allow pipelined and partitioned parallelism

Key term: Partitioned parallelism — Split an operator's input over many processors and run a copy of the operator on each part at once

Required

Partition the Data So Each Node Works on Its Part

WHAT TO TAKE AWAY

1. Start-up, interference and skew are the three barriers to linear speedup and scaleup
2. With 1% interference per added processor, speedup can never exceed 37
3. Round-robin suits full scans, hashing suits key lookups, ranges suit range queries but risk skew

THIS WEEK

Builds on Lesson 4-1's shared-nothing slide. In Practice 4, sites split customers by state, a value-based scheme close to his range partitioning; Week 5 studies partitioning.

LIMITATION

A 1992 survey: its systems (Teradata, Tandem, Gamma, Bubba) are now history, and its second half lists open problems rather than results.

For your half-page note: Define speedup and scaleup with your own example. Which partitioning would suit lookups by customer number, and why?

Required

Autonomy Separates Federations From Distributed DBMSs

Sheth, A. P., & Larson, J. A. (1990). Federated database systems for managing distributed, heterogeneous, and autonomous databases. ACM Computing Surveys, 22(3), 183–236. https://doi.org/10.1145/96602.96604

THE PROBLEM

Organisations ran many separate databases, often on different DBMSs, and needed to share data across them without losing control. Sheth and Larson survey how to build federations of such autonomous, heterogeneous databases.

KEY IDEAS

01 Three dimensions describe a multidatabase system: distribution, heterogeneity and autonomy

02 Autonomy has several kinds: design, communication, execution and association

03 Taxonomy: non-federated or federated; a federation is loosely or tightly coupled

Key term: Federated system — A set of cooperating component databases that stay autonomous but agree to share part of their data

Required

Who Controls the Federated Schema Decides the Type

WHAT TO TAKE AWAY

1. Heterogeneity comes from different DBMSs, data models and languages, and from what data means
2. Tightly coupled: administrators build a federated schema; loosely coupled: users build views
3. Components keep control of their data, so full transparency is traded for autonomy

THIS WEEK

Lesson 4-2's 'Three Independent Dimensions' and 'Distributed DBMS or Multi-Database System?' use this survey. In Practice 4 Part C you rate 8 systems on A, D and H.

LIMITATION

Written in 1990, before the Web and cloud data services. At 54 pages it is long, and its later sections on schema translation are dated.

For your half-page note: Using the kinds of autonomy, explain why a federation of independent databases cannot offer full transparency.

Optional

SDD-1: An Early Working Distributed DBMS

Rothnie, J. B., Bernstein, P. A., Fox, S., Goodman, N., Hammer, M., Landers, T. A., Reeve, C., Shipman, D. W., & Wong, E. (1980). Introduction to a system for distributed databases (SDD-1). ACM Transactions on Database Systems, 5(1), 1–17. https://doi.org/10.1145/320128.320129

THE PROBLEM

In the late 1970s, data was spread over sites on a network, and no DBMS could manage it as one database. The SDD-1 team at Computer Corporation of America built one, and had to solve query processing, concurrency and failures.

KEY IDEAS

01 Relations are split into fragments, and each fragment may be stored at several sites

02 Three kinds of virtual machine: Transaction Modules, Data Modules and a Reliable Network

03 Users see one relational database; SDD-1 hides where fragments and copies are stored

Key term: Data Module — The SDD-1 component at each site that stores fragments and carries out requests to read, move and write data

Optional

Reduce Data With Semi-Joins, Then Ship It

WHAT TO TAKE AWAY

1. Queries reduce relations with semi-joins before moving them to the site that finishes the work
2. Concurrency control uses timestamps and transaction classes analysed in advance for conflicts
3. The Reliable Network gives guaranteed delivery, atomic commit, site monitoring and a clock

THIS WEEK

Lesson 4-1's 'Problems Every Distributed DBMS Must Solve' appear here in a real system. Its query processing is what Lesson 4-3's 'Why the Semi-Join Matters: Ship Fewer Bytes' explains.

LIMITATION

A 1980 prototype on the slow ARPANET. Its design assumes communication cost dominates, which is less true on today's networks.

To think about: SDD-1 assumed the network was the bottleneck. Which of its design choices would you change for a fast data-centre network?

Optional

Semi-Joins Fully Reduce Only Tree Queries

Bernstein, P. A., & Chiu, D.-M. W. (1981). Using semi-joins to solve relational queries. Journal of the ACM, 28(1), 25–40. https://doi.org/10.1145/322234.322238

THE PROBLEM

In a distributed database, joining relations at different sites means shipping data, and communication is the main cost. Semi-joins can shrink relations first. But which queries can semi-joins fully reduce, and how many are needed?

KEY IDEAS

01 A semi-join keeps the tuples of R that match some tuple of S; only S's join column is sent

02 A query graph has a node per relation and an edge per join clause

03 Full reduction removes every tuple that does not take part in the query's answer

Key term: Semi-join — The join of R and S projected back onto R's columns: the tuples of R that match some tuple of S

Optional

Cycles in the Query Graph Defeat Semi-Joins

WHAT TO TAKE AWAY

1. Theorem 1: a tree query on n relations is fully reduced by a program of 2n – 2 semi-joins
2. Theorem 2: for cyclic queries, the semi-joins needed can grow with relation size, not query size
3. Theorem 3: whether a query is a tree query can be tested in linear time

THIS WEEK

The formal basis of Lesson 4-3's 'Why the Semi-Join Matters: Ship Fewer Bytes'. In Practice 4 Part B, pushdown ships only rows that pass a filter; a semi-join does this for join values.

LIMITATION

Pure theory: it counts semi-joins but not their cost, and leaves out selections, target lists and multi-column semi-joins.

To think about: Draw the query graph of a three-table join from Practice 4. Is it a tree query, and what does that mean for a semi-join plan?

Optional

A Distributed DBMS Makes Many Sites Look Like One

Özsu, M. T., & Valduriez, P. (2020). Introduction. In Principles of distributed database systems (4th ed., pp. 1–31). Springer. https://doi.org/10.1007/978-3-030-26253-2_1

THE PROBLEM

Data is now spread over many computers, from company networks to clouds. Chapter 1 of the standard textbook defines a distributed database, explains what a distributed DBMS promises, and lists the problems it must solve.

KEY IDEAS

01 A distributed database is logically related data spread over sites linked by a network

02 Promises: transparency, reliability through distributed transactions, performance, scalability

03 Architectures are classified by autonomy, distribution and heterogeneity

Key term: Distributed DBMS — The software that manages a distributed database and makes the distribution transparent to its users

Optional

The Problems Behind Every Distributed Database

WHAT TO TAKE AWAY

1. Design issues include data placement, query processing, concurrency control and reliability
2. Transparency hides where data is stored, how it is fragmented and whether it is copied
3. Main architectural models: client–server, peer-to-peer, multidatabase and cloud

THIS WEEK

Covers the whole week: Lesson 4-1's definitions and Lesson 4-2's transparency and A/D/H dimensions. Use its terms for the reflection on ddb_global in Practice 4.

LIMITATION

A textbook chapter, not research: it gives definitions and an overview without new results, and its examples change between editions.

To think about: Which promises of a distributed DBMS does Practice 4's global layer keep, and which does it not keep?

Questions

Reading Questions

Try to answer each question in two or three sentences before the next class.

Q1	Why does shared nothing scale better than shared memory and shared disk, and which problem does it leave to the designer?	Stonebraker
Q2	What are speedup and scaleup, and which three barriers keep speedup from being linear?	DeWitt, Gray
Q3	How do distribution, heterogeneity and autonomy differ, and what is a tightly coupled federation?	Sheth, Larson
Q4	Classify the Practice 4 setup and a Citus cluster using both Sheth and Larson's terms and A/D/H.	All papers

WEEK 4 · WRAP-UP

This Week’s Work

Practice 4

Lab report

Algebra, a two-site database and the A/D/H worksheet. One PDF, at most 4 pages, 10 points.

Due: before Week 5

Quiz 4

Distributed databases

Lessons 4-1 to 4-3: 16 multiple-choice questions and one short design question, 30 minutes.

Due: before Week 5

Reading

Required papers

Stonebraker, DeWitt and Gray, Sheth and Larson. A half-page note for each paper.

Bring to Week 5

Next week: which rows go to which site? Horizontal fragmentation, designed from the queries people actually run.